

IoT-Enabled Retail Analytics on Azure Databricks

Resources Creation

creation of storage account:

The screenshot displays the Azure portal interface for a storage account named 'sagarcatalog'. The left sidebar shows the 'Storage accounts' section with a list of accounts: 'databricksmentor', 'nilaystore', 'sagarcatalog' (selected), and 'sagarstorage0'. The main content area shows the 'Overview' tab for 'sagarcatalog'. It includes a search bar, a list of actions (Upload, Open in Explorer, Delete, Move, Refresh, Open in mobile, CLI / PS, Feedback), and a 'JSON View' link. The 'Essentials' section lists key properties: Resource group (westus2), Location (westus2), Subscription (LabKraft), Subscription ID (d008c7cc-9795-4292-bf79-74ae52cda63d), Disk state (Available), and Tags (Add tags). The 'Properties' section shows 'Data Lake Storage' settings: Hierarchical namespace (Enabled), Default access tier (Hot), and Blob anonymous access (Disabled). The 'Security' section shows 'Require secure transfer for REST API operations' (Enabled) and 'Storage account key access' (Enabled).

creation of access connector:

The screenshot displays the Azure portal interface for an access connector named 'sagarconnector'. The left sidebar shows the 'Access Connect...' section with a list of connectors: 'sagarconnector' (selected). The main content area shows the 'Overview' tab for 'sagarconnector'. It includes a search bar, a list of actions (Refresh, Delete), and a 'JSON View' link. The 'Essentials' section lists key properties: Resource group (westus2), Location (West US 2), Subscription (LabKraft), Subscription ID (d008c7cc-9795-4292-bf79-74ae52cda63d), and Tags (Add tags). The 'Properties' section shows 'Data Lake Storage' settings: Hierarchical namespace (Enabled), Default access tier (Hot), and Blob anonymous access (Disabled). The 'Security' section shows 'Require secure transfer for REST API operations' (Enabled) and 'Storage account key access' (Enabled).

giving access connector permission for storage:

Microsoft Azure

Search resources, services, and docs (G+/I)

Copilot

fredence14@michaelklo...
DEFAULT DIRECTORY (MICHAELK...

Home > Storage accounts > sagarstorage0 | Access Control (IAM) >

Add role assignment

RoleMembersConditionsReview + assign

Showing a filtered list of roles because your permissions include a condition. [Learn more](#)
[View my access](#)

Selected role

Storage Blob Data Contributor

Assign access to

User, group, or service principal

Managed identity

Members

+ Select members

Name	Object ID	Type
No members selected		

Description

Optional

Review + assign

Previous

Next

Select managed identities

Some results might be hidden due to your ABAC condition.

Subscription *

LabsKraft

Managed identity

Access Connector for Azure Databricks (1)

Search by name

sagarcconnector
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/LabsKra...

Selected members:

No members selected. Search for and add one or more members you want to assign to the role for this resource.

[Learn more about RBAC](#)

Select

Close

Feedback

creation of uc:

Microsoft Azuredatabricks

Search data, notebooks, recents, and more...

CTRL + P

tred_d4

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

AI/ML

Playground

Experiments

Features

Catalog

Serverless Starter Warehouse Serverless S

Type to search...

My organization

tred_d4

system

sagar_cat_project1

default

dit_output

information_schema

sagar_catalog

sagar_catalog2

sagar_catalog3

pratcatalog

pratcatalog1

sahil_catalog

sahil_catalog_new

sanyam_catalog

sanyam_new_catalog

Delta Shares Received

samples

Legacy

hive_metastore

Catalog Explorer

sagar_cat_project1

Share

Create schema

OverviewDetailsPermissionsWorkspaces

About this catalog

Metastore Id

4f3b2018-fdc7-47a3-965c-db7d2dc65f0b

Created At

Aug 25, 2025, 09:13 AM

Created By

tredence14@michaelkloudkraft@hotmail.onmicrosoft.com

Updated At

Aug 25, 2025, 09:13 AM

Updated By

tredence14@michaelkloudkraft@hotmail.onmicrosoft.com

Storage Root

abfss://raw0@sagarcatalog.dfs.core.windows.net/

Storage Location

abfss://raw0@sagarcatalog.dfs.core.windows.net/_unitystorage/catalogs/00d8640c-c495-4e5e-b9c6-447107a58745

Advanced

Predictive Optimization

ENABLED (Inherited)

creation of namespace and eventhub:

[Home](#) > [Event Hubs](#) >

 **Create Namespace** ...

Event Hubs

Project Details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

LabsKraft

Resource group *

LabsKraft2027

[Create new](#)

Instance Details

Enter required settings for this namespace, including a price tier and configuring the number of units (capacity).

Namespace name *

sagarspace ✓
.servicebus.windows.net

Region *

West US 2

 The region selected supports Availability zones. Your namespace will have Availability Zones enabled. [Learn more.](#)

Pricing tier *

Basic

[Browse the available plans and their features](#)

Throughput Units *

1

[Review + create](#)

< Previous

Next: Advanced >

Microsoft Azure

Search resources, services, and docs (G+/I)

Copilot

tredence14@michaelklo...
DEFAULT DIRECTORY (MICHAELKLO...)

Home > Event Hubs >

Create Namespace

Event Hubs

Validating...

Basics Advanced Networking Tags Review + create

Event Hubs Namespace
by Microsoft

Basics

Namespace name	sagarhub
Subscription	LabsKraft
Resource group	LabsKraft2027
Region	West US 2
Pricing tier	Basic
Throughput Units	1
Availability Zones (Zone Redundancy)	Enabled

Networking

Connectivity method	Public access
---------------------	---------------

Security

Minimum TLS version	1.2
---------------------	-----

Create < Previous Next >

24°C
Mostly cloudy

Search

ENG
IN

10:48
25-08-2025

Microsoft Azure

Search resources, services, and docs (G+/I)

Copilot

tredence14@michaelklo...
DEFAULT DIRECTORY (MICHAELKLO...)

Home > Event Hubs > sagarspace >

Create Event Hub

Event Hubs

Validation succeeded.

Basics Capture Review + create

Event Hubs Instance
by Microsoft

Basics

Name	sagarhub
Partition count	1

Retention

Cleanup policy	Delete
Retention time (hrs)	6

Capture

Capture Status	Not Supported
----------------	---------------

Create < Previous Next >

creating synapse workspace:

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

DEFACTO GROUP
DEFAULT DIRECTORY

Home > Azure Synapse Analytics >

Create Synapse workspace ...

Validation succeeded

information with the provider(s) of the offering(s) for support, billing and other transactional activities. Microsoft does not provide rights for third-party offerings. For additional details see [Azure Marketplace Terms](#).

Basics

Subscription

Resource group

Region

Workspace name

Data Lake Storage Gen2 account

Data Lake Storage Gen2 file system

Managed resource group

Role assignments

LabsKraft

LabsKraft2027

West US 2

(new) sagars

https://sagarstorage0.dfs.core.windows.net

raw

None

The Storage Blob Data Contributor role will be assigned on the specified Data Lake Storage Gen2 account to both the workspace managed identity and the current user.

Security

Authentication method

SQL Server admin login

SQL Password

Double encryption

Use both local and Microsoft Entra ID authentication

sqladminuser

No

Create

< Previous

Next >

Download a template for automation

setting up synapse ->
linking my adls for storing old data and these can be pulled into azure dbx

Microsoft Azure

Synapse Analytics

sagars

DEFACTO GROUP
DEFAULT DIRECTORY

Synapse live

Validate all

Publish all

Data

Workspace

Linked

Filter resources by name

Azure Data Lake Storage Gen2 2

sagars (Primary - sagarstorage0)

raw (Primary)

(Attached Containers)

raw

Other users in your workspace may have access to modify this item. Do not use this item unless you trust all users who may have access to the workspace.

New SQL script

New notebook

New data flow

New integration dataset

Upload

Download

New folder

More

raw

Name	Last Modified	Content Type	Size
retail_iot_data.csv	8/25/2025, 10:26:07 AM		130.8 KB

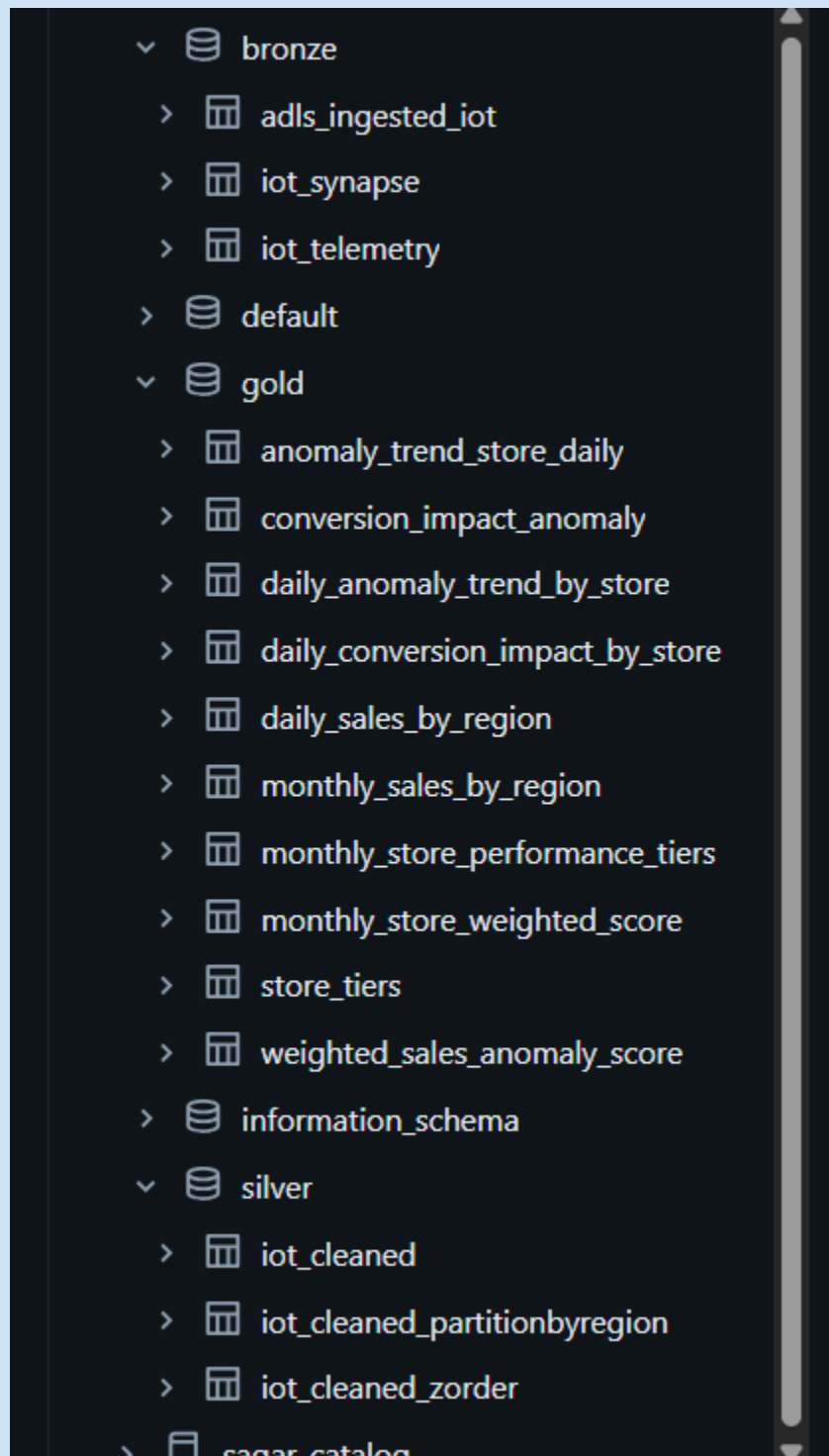
creation of new dedicated pool:

The screenshot shows the Azure Synapse Analytics interface for creating a new dedicated SQL pool. The left sidebar contains navigation options like 'SQL pools', 'Apache Spark pools', and 'Data Explorer pools'. The main content area is titled 'New dedicated SQL pool' and shows a 'Validation succeeded' message. The 'Review + create' tab is selected, displaying the following information:

- Product details:** Azure Synapse Analytics dedicated SQL pool by Microsoft. Estimated cost per hour is 99.83 INR.
- Terms:** A legal disclaimer stating that by clicking 'Create', the user agrees to the legal terms and privacy statement(s) associated with the Marketplace offering(s).
- Data source:** Dedicated SQL pool name is 'db1' and Performance level is 'DW100c'.
- Additional settings:** Use existing data is 'Blank' and Collation is 'SQL_Latin1_General_CP1_CI_AS'.

At the bottom, there are buttons for 'Create', '< Previous', 'Download template for automation', and 'Cancel'.

Quick Summary->



Ingestion mode

i) ADLS

adls connection and storing file in sagar_cat_project1.bronze schema

The screenshot displays the Databricks workspace interface. On the left, the 'Catalog' sidebar shows a tree structure with 'My organization' at the top, followed by 'tred_d4', 'system', and 'sagar_cat_project1'. Under 'sagar_cat_project1', the 'bronze' schema is selected. The main workspace area shows a notebook with four cells. The first cell is a SQL statement: `use schema bronze;`. The second cell is a comment: `# Correct ADLS Gen2 config (dfs, not blob)`. The third cell is a Python code block that sets the ADLS Gen2 configuration: `storage_account_name='sagarstorage0'`, `storage_account_key='M97WMTtMgBp9d2w6dFHzTG00uYIDEun7x6cnpaA2PB7x/zfa59Jc0iIVIn8WnA4S8VnMZUn68y+ASTMF8+Xg=='`, and `spark.conf.set(f'fs.azure.account.key.{storage_account_name}.dfs.core.windows.net', storage_account_key)`. The fourth cell is a Python code block that reads data from ADLS Gen2: `adls_ingested=(spark.read.format("csv").option("header", "true").option("inferSchema", "true").load("abfss://raw@sagarstorage0.dfs.core.windows.net/retail_iot_data.csv"))` and writes it to the 'adls_ingested_iot' table: `adls_ingested.write.mode("overwrite").saveAsTable("adls_ingested_iot")`. The right sidebar shows the user 'Tredence Labs/Kraft14' and the email 'tredence14@michaelkoudkrathotmail.onmicrosoft.com'.

adls_ingested_table->

The screenshot shows the Microsoft Azure Databricks interface. The top bar includes the Microsoft Azure logo, the Databricks logo, a search bar, and a user profile icon. The left sidebar contains navigation icons for Home, Workspace, Recent, and Clusters. The main area displays a Python notebook with the following code:

```
.option("inferSchema", "true")\
.load("abfss://raw@sagarstorage0.dfs.core.windows.net/retail_iot_data.csv")\

adls_ingested.write.mode("overwrite").saveAsTable("adls_ingested_iot")\
display(adls_ingested)
```

Below the code, the output shows a table with 14 rows and 12 columns. The columns are: OrderID, CustomerID, Region, StoreID, Product, Quantity, UnitPrice, SalesAmount, OrderTime, and DeviceID. The data represents retail IoT transactions.

OrderID	CustomerID	Region	StoreID	Product	Quantity	UnitPrice	SalesAmount	OrderTime	DeviceID
db07ecdd-e8e7-408a-846e-c2d330484d4b	CUST-2824	East	Store-009	Mobile	2	322.1	644.2	2023-01-05T23:24:55.000+00...	Temp_Sc
93d89258-6e02-4bb8-9a71-cb590cb55f85	CUST-7912	East	Store-001	Laptop	2	503.69	1007.38	2023-01-30T05:24:09.000+00...	RFID_Sc
f97636e9-3dab-48ad-aad4-0ca034d31111	CUST-9928	South	Store-008	Tablet	5	592.47	2962.35	2023-01-01T07:34:17.000+00...	RFID_Sc
120aef76-3e09-4580-8429-af1d6b76209a	CUST-3547	West	Store-011	Laptop	1	790.86	790.86	2023-01-18T10:14:28.000+00...	POS_Ter
5dbcd9dd-0275-4427-a4b6-089b43465200	CUST-8527	East	Store-013	Laptop	11	621.7	6838.7	2023-01-18T13:20:07.000+00...	Temp_Sc
c6bd3963-d239-42b7-b414-4e39289bc2da	CUST-5741	East	Store-008	Printer	1	791.25	791.25	2023-01-23T00:15:40.000+00...	Temp_Sc
03dad02-d15e-4527-94b1-9cb915a71209	CUST-6820	West	Store-009	Smartwatch	1	1237.81	1237.81	2023-01-09T07:22:27.000+00...	Temp_Sc
90e7e190-9d09-47a3-a399-985c2cf389dc	CUST-7216	North	Store-018	Mobile	3	1693.56	5080.68	2023-01-03T17:10:20.000+00...	RFID_Sc
50f6e840-47fc-4117-a776-78701a911199	CUST-7572	North	Store-003	Mobile	5	1758.92	8794.6	2023-01-16T06:37:34.000+00...	RFID_Sc
77d4cd70-6019-4be0-bce4-51626ae52101	CUST-8517	West	Store-009	Mobile	2	1502.73	3005.46	2023-01-27T03:57:14.000+00...	POS_Ter
6d65d67c-bc5e-43bf-9522-50e645a5f8f1	CUST-7543	North	Store-008	Mobile	5	1012.36	5061.8	2023-01-03T06:53:40.000+00...	RFID_Sc
d6c3b3f1-076e-4c24-a836-318d6588eedd	CUST-7916	East	Store-013	Tablet	5	1992.44	9962.2	2023-01-26T16:27:46.000+00...	POS_Ter
18175f8f-6017-413a-9e20-c876857ee103	CUST-1188	East	Store-018	Printer	3	1548.77	4646.31	2023-01-17T12:19:59.000+00...	RFID_Sc
54e45330-e78f-4d3d-9426-8eb38292bbd3	CUST-1053	North	Store-017	Printer	2	1039.98	2079.96	2023-01-06T03:58:39.000+00...	Temp_Sc

ii)Event Hub:

we will connect to vm and then send data from local machine to eventhub which at the end will be recieved by DBX

creation of vm (namespace and eventhub already created) ->

The screenshot shows the Microsoft Azure portal interface for creating a virtual machine. The top bar includes the Microsoft Azure logo, a search bar, and a user profile icon. The left sidebar contains navigation icons for Home, Compute infrastructure, Virtual machines, and other resources. The main area displays the 'Create a virtual machine' wizard with the following steps:

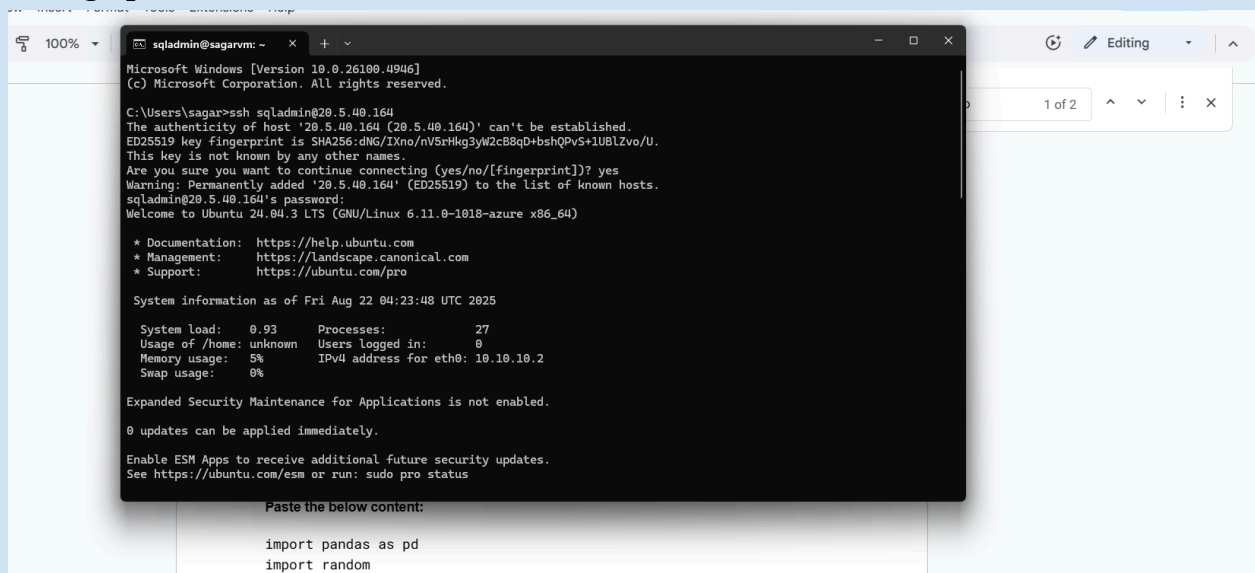
- Validation passed
- Enter e-mail address
- Preferred phone number
- Basics

The 'Basics' section shows the following configuration:

- Subscription: LabsKraft
- Resource group: LabsKraft2027
- Virtual machine name: sagarvm
- Region: Australia East
- Availability options: Availability zone
- Zone options: Self-selected zone
- Availability zone: 1
- Security type: Trusted launch virtual machines
- Enable secure boot: Yes
- Enable vTPM: Yes
- Integrity monitoring: No

At the bottom, there are buttons for '< Previous', 'Next >', and 'Create'. A link for 'Download a template for automation' and a 'Give feedback' button are also present.

setting up vm session on local machine->



```
Microsoft Windows [Version 10.0.26100.4946]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sagar>ssh sqladmin@20.5.40.164
The authenticity of host '20.5.40.164 (20.5.40.164)' can't be established.
ED25519 key fingerprint is SHA256:dNG/IXno/nV5rHkg3yW2cB8qD+bshqPvS+1UBLZvo/U.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '20.5.40.164' (ED25519) to the list of known hosts.
sqladmin@20.5.40.164's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.11.0-1018-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Fri Aug 22 04:23:48 UTC 2025

System load:  0.93      Processes:    27
Usage of /home: unknown  Users logged in:  0
Memory usage: 5%       IPv4 address for eth0: 10.10.10.2
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

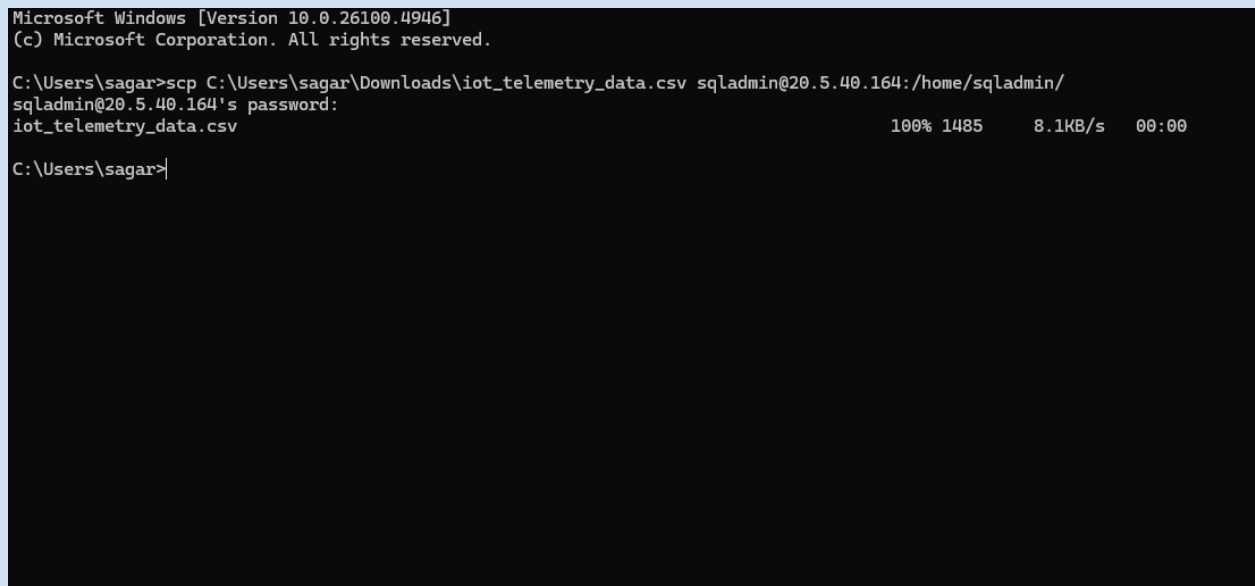
0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Paste the below content:

import pandas as pd
import random
```

uploading my file to vm which is considered for ingestion



```
Microsoft Windows [Version 10.0.26100.4946]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sagar>scp C:\Users\sagar\Downloads\iot_telemetry_data.csv sqladmin@20.5.40.164:/home/sqladmin/
sqladmin@20.5.40.164's password:
iot_telemetry_data.csv                                100% 1485      8.1KB/s   00:00

C:\Users\sagar>
```

nano [send.py](#)

```
GNU nano 7.2 send.py
ls /home/sqladmin/ from azure.eventhub import EventHubProducerClient, EventData
import pandas as pd
import time
import json

# Load the CSV
df = pd.read_csv('hot_telemetry_data.csv')

# Event Hub connection
connection_str = "Endpoint=sb://sagarspace.servicebus.windows.net/;SharedAccessKeyName=RootManageSharedAccessKey;SharedAccessKey=2v87F8+tBbuM02M5qjzsPlUAVWbYgfeZ+AEhBrynDE="
eventhub_name = "sagarhub"

producer = EventHubProducerClient.from_connection_string(conn_str=connection_str, eventhub_name=eventhub_name)

# Stream data row by row
for _, row in df.iterrows():
    data = row.to_dict()
    event_data_batch = producer.create_batch()
    event_data_batch.add(EventData(json.dumps(data)))
    producer.send_batch(event_data_batch)
    print(f'Sent: {data["OrderID"]}')
    time.sleep(1) # simulate 1 event/sec
```

sent file:

```
WARNING: The script faker is installed in '/home/sqladmin/.local/bin' which is not
Consider adding this directory to PATH or, if you prefer to suppress this warning,
Successfully installed faker-37.5.3 numpy-2.3.2 pandas-2.3.2 tzdata-2025.2
sqladmin@sagarvm:~$ python3 send.py
Sent: a1f3d6b7-45e8-4a30-8e9b-1f9d2caa6710
Sent: b7d4f3c2-9c23-4a0a-a4b5-f0317b53f8f5
Sent: c6e8f5a1-58a9-4980-a71c-34266c5db412
Sent: d8f4c9b2-92d2-4a9f-a6b0-15db9d214e62
Sent: e3c9a2f1-73a4-41e8-b4fd-62b0a930ed7b
Sent: f5d6b8c3-84b9-4a77-a9c1-9d1e1cbf6d2e
Sent: g7a2d1e4-91c5-4f21-bf28-7bc0e55d9183
Sent: h9f8e3d5-11a2-48a4-9c2b-64b0b11c6d45
Sent: i2d4b7a6-5c31-4c29-bd38-27f7a04d0c9e
```

File received on EH

The screenshot shows the Azure Event Hubs Data Explorer for an instance named 'sagarhub'. The left sidebar contains navigation options: Overview, Access control (IAM), Diagnose and solve problems, Data Explorer (selected), Resource visualizer, Settings, Entities, Features, Automation, and Help. The main area displays the 'sagarhub' instance with a 'Send events' button and a table of received events. The table has columns for Sequence Number, Offset, Partition ID, Enqueued Time, Content Type, Message ID, and Event Body. Below the table, there are tabs for 'Event body' and 'Event properties'. The 'Event body' tab is active, showing a scrollable list of event bodies. The 'Event properties' tab is also visible.

Sequence Nu...	Offset	Partition ID	Enqueued Time	Content Type	Message ID	Event Body
0	0	0	Mon, Aug 25, 25, 12:05:40 PM G...			{"OrderID": "a1f3d6b7-45e8"
1	424	0	Mon, Aug 25, 25, 12:05:42 PM G...			{"OrderID": "b7d4f3c2-9c23"
2	840	0	Mon, Aug 25, 25, 12:05:43 PM G...			{"OrderID": "c6e8f5a1-58a9"
3	1264	0	Mon, Aug 25, 25, 12:05:44 PM G...			{"OrderID": "d8f4c9b2-92d2"
4	1688	0	Mon, Aug 25, 25, 12:05:45 PM G...			{"OrderID": "e3c9a2f1-73a4"
5	2112	0	Mon, Aug 25, 25, 12:05:46 PM G...			{"OrderID": "f5d6b8c3-84b9"
6	2552	0	Mon, Aug 25, 25, 12:05:48 PM G...			{"OrderID": "g7a2d1e4-91c5"
7	2968	0	Mon, Aug 25, 25, 12:05:49 PM G...			{"OrderID": "h9f8e3d5-11a2"
8	3384	0	Mon, Aug 25, 25, 12:05:50 PM G...			{"OrderID": "i2d4b7a6-5c31"
9	3800	0	Mon, Aug 25, 25, 12:05:51 PM G...			{"OrderID": "j3e9c5b8-7444"

Reading data from eventhub and storing into my bronze schema as `iot_telemetry`

The screenshot shows a Databricks workspace with a Python script in the editor. The script defines a schema for 'iot_telemetry' and reads data from an Event Hub using Spark's `readStream` method. The data is then written to a table named 'iot_telemetry' using `write.mode('append').saveAsTable('iot_telemetry')`.

```
from pyspark.sql.types import *
from pyspark.sql.functions import *

schema = StructType([
    StructField("OrderID", StringType()),
    StructField("CustomerID", StringType()),
    StructField("Region", StringType()),
    StructField("StoreID", StringType()),
    StructField("Product", StringType()),
    StructField("Quantity", IntegerType()),
    StructField("UnitPrice", DoubleType()),
    StructField("SalesAmount", DoubleType()),
    StructField("OrderTime", StringType()),
    StructField("DeviceType", StringType()),
    StructField("Temperature", DoubleType()),
    StructField("DeviceStatus", StringType()),
    StructField("AnomalyFlag", IntegerType()),
    StructField("AnomalyType", StringType())
])

stream_df = (
    spark.readStream
        .format("eventhubs")
        .options(**eh_conf)
        .load()
)

telemetry_df = (
    stream_df
        .selectExpr("cast(body as string) as json")
        .select(from_json("json", schema).alias("data"))
        .select("data.*")
)

telemetry_df.write.mode('append').saveAsTable('iot_telemetry')
```

error i got->

The screenshot shows a Databricks error message: `[UNSUPPORTED_STREAMING_SOURCE_PERMISSION_ENFORCED] Data source eventhubs is not supported as a streaming source on a shared cluster. SQLSTATE: 0A000`. The error message is displayed in a red box with a warning icon.

i solved by making personal cluster .

Data streamed in from event_hub->

The screenshot displays the Microsoft Azure Databricks interface. On the left, the 'Workspace' sidebar shows a catalog with various databases and tables. The main area shows a Python notebook with a query that reads from a table named 'telemetry' and writes the results to a new table named 'iot_telemetry'. Below the notebook, a table view is shown with columns: OrderID, CustomerID, Region, StoreID, Product, Quantity, and UnitPrice. The table contains 24 rows of data.

```
telemetry_df = (
    stream_df
    .selectExpr("cast(body as string) as json")
    .select(from_json("json", schema).alias("data"))
    .select("data.*")
)

telemetry_df.write.mode('append').saveAsTable('iot_telemetry')
display(telemetry_df)
```

OrderID	CustomerID	Region	StoreID	Product	Quantity	UnitPrice
77d4cd70-6019-4be0-bce4-51626ae52101	CUST-8517	West	Store-009	Mobile	2	1502.73
6d85d67c-bc5e-43bf-9522-50e645fa5ff4	CUST-7543	North	Store-008	Mobile	5	1012.36
d8c2b3f1-076e-4c24-a836-318d9388eedd	CUST-7916	East	Store-013	Tablet	5	1992.44
18179bf8-6017-413a-9e20-c876037ee103	CUST-1188	East	Store-018	Printer	3	1548.77
54a45330-e78f-4d3d-9426-8eb38293bbd3	CUST-1083	North	Store-017	Printer	2	1039.88
f40e3d98-1113-4b72-9a70-3e1dcaa00544	CUST-4258	West	Store-012	Printer	2	1101.79
f8d9eb0c-6447-4c10-8e46-cbb6a04d94e4	CUST-2832	North	Store-010	Mobile	1	519.7
1131c559-e455-4c4d-91c2-d5754ab7ad1d	CUST-2133	West	Store-005	Smartwatch	4	1896.35
307e7b0c-128e-46e0-add0-89fa1ff2eea3	CUST-4470	West	Store-010	Tablet	3	904.3
a4efe082-f0d2-40cc-8b05-07d3f3259e94	CUST-6539	East	Store-019	Camera	2	1197.46
ccc9fab-748e-43f5-ba0d-ec19386d44c0	CUST-6413	East	Store-017	Mobile	3	1354.51
a19e0f19-3461-45de-b5e7-f706152d2afc	CUST-8744	West	Store-016	Printer	4	421.3
6669123b-765f-420a-90c2-631e8fc3f3f0	CUST-8651	East	Store-004	Laptop	4	1470.82
f349ff6-2650-4e07-b1eb-a14d7d8b1922	CUST-3296	South	Store-006	Headphones	4	537.12

Both table from adls and eventhub stored in my bronze schema of catalog.

iii) Azure synapse ingestion ->

```
# Synapse connection details
synapse_workspace = "sagars"
synapse_pool = "db1" # Your dedicated SQL pool name
synapse_user = 'sqladmin'
synapse_password = ''
synapse_jdbc_url = f"jdbc:sqlserver://{synapse_workspace}.sql.azuresynapse.net:1433;database={synapse_pool};"

# Temporary staging location on ADLS Gen2
# This MUST be a storage account accessible by both Synapse and Databricks
temp_storage_account = "sagarstorage08"
temp_container = "syn"
temp_dir = f"abfss://{temp_container}@{temp_storage_account}.dfs.core.windows.net/synapse-dbx-staging"

# Source and Target table names
source_table = "dbo.RetailSales"
target_catalog = "sagar_cat_project1"
target_schema = "bronze"
target_table_name = f"{target_catalog}.{target_schema}.iot_syn"

# 2. Read from Synapse using the connector
print(f"Reading data from Synapse table: {source_table}...")
df = (spark.read
      .format("com.microsoft.sqlserver.jdbc.spark") # Use the Synapse connector
      .option("url", synapse_jdbc_url)
      .option("dbtable", source_table)
      .option("user", synapse_user)
      .option("password", synapse_password)
      .option("tempDir", temp_dir) # Crucial for performance
      .option("forwardSparkAzureStorageCredentials", "true")
      .load())

# 3. Write the data as a Delta table in Databricks
print(f"Writing data to Delta table: {target_table_name}...")
(df.write
  .format("delta")
  .mode("overwrite")
  .saveAsTable(target_table_name))

print("Import complete!")
```

display(spark.table('sagar_cat_project1.bronze.iot_syn'))

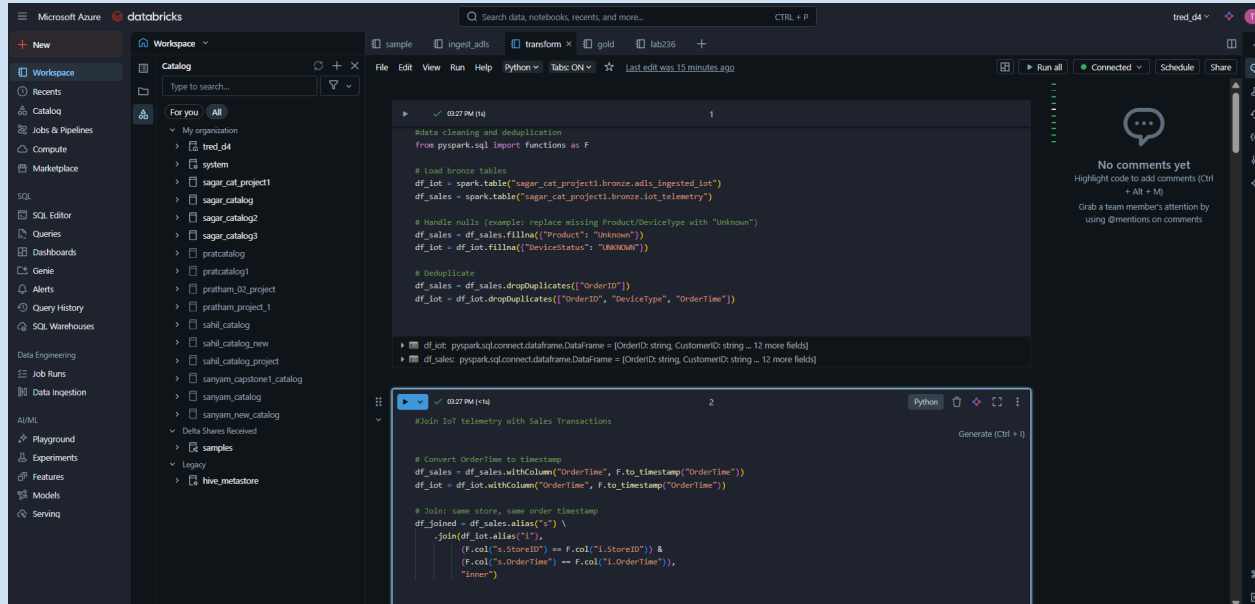
► (4) Spark Jobs

► adls_ingested: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 12 more fields]

	OrderID	CustomerID	Region	StoreID	Product	Quantity
57	9a72be7d-3ca5-4b64-878c-f443aea6cae5	CUST-4613	South	Store-008	Headphones	4
58	9fad8b36-fcd2-4037-a019-728e16ca3686	CUST-3704	South	Store-005	Camera	13
59	199de6e5-8d11-4887-b2b7-c05dab1b01...	CUST-3223	South	Store-006	Laptop	3
60	8b67fa52-42bb-486a-866c-aa1a893ee151	CUST-7211	North	Store-014	Headphones	1
61	3afae91f-cc69-46ec-941b-58cefc85d7e	CUST-2124	East	Store-001	Mobile	2
62	fae848cf-90c7-46fe-af09-87e58efbf673	CUST-4571	South	Store-009	Printer	3
63	14dc706f-5b8c-45ff-98a3-5246959b1b69	CUST-3683	North	Store-004	Camera	1
64	2b4da48a-9939-4bb3-8203-0f35369dff71	CUST-4249	East	Store-019	Smartwatch	2
65	2bb8c321-2f07-408d-895a-217cda90ee50	CUST-1672	North	Store-018	Tablet	3
66	82961f57-063d-452e-abe1-c8c0ce076f1d	CUST-2729	South	Store-012	Smartwatch	4
67	cd669277-e660-44cd-82ad-a365c6d375cd	CUST-5425	South	Store-015	Tablet	5
68	22c7aca4-9dda-4759-b9b9-2c585ae3cc71	CUST-8613	South	Store-011	Laptop	4
69	37ce4b04-72d4-4b8e-ad0f-0d98e56445a0	CUST-6576	North	Store-020	Smartwatch	3
70	5e8bc1da-b338-4da5-a533-e79d55161bc9	CUST-9004	West	Store-016	Smartwatch	4
71						

Transformation:

Removal of data quality issues and renaming column



The screenshot shows a Databricks workspace with a notebook titled 'tred_d4'. The notebook contains the following code:

```
#data cleaning and deduplication
from pyspark.sql import functions as F

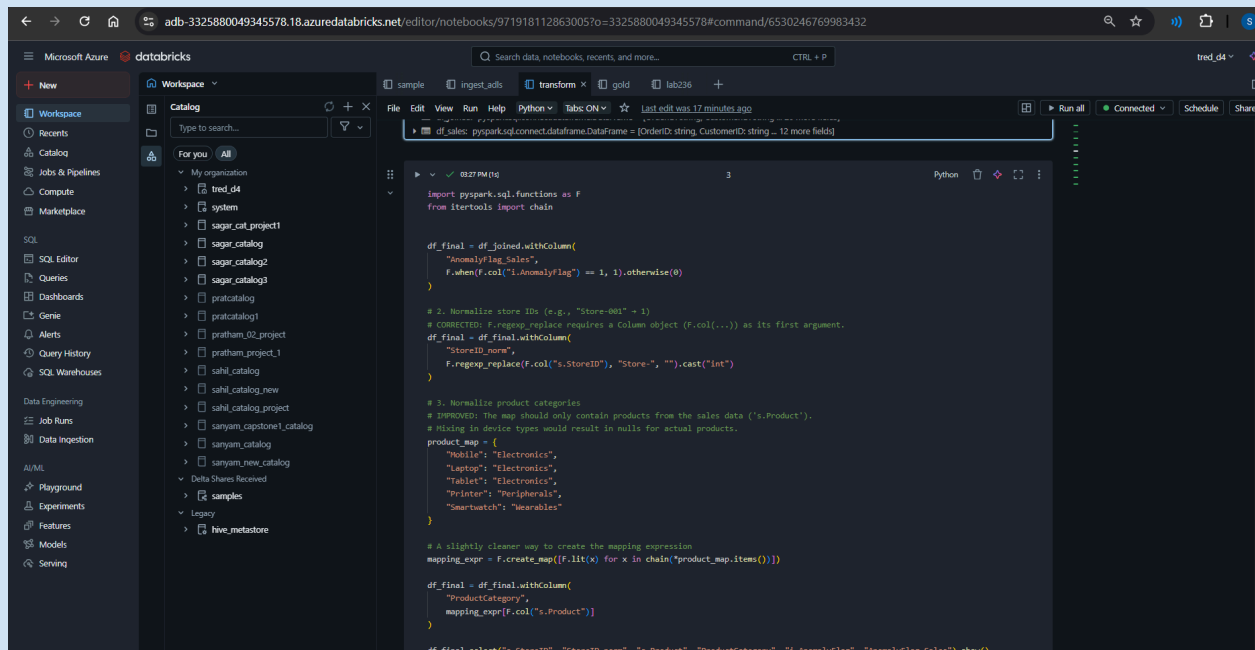
# Load bronze tables
df_iot = spark.table("sagar_cat_project1.bronze.adls_ingested_iot")
df_sales = spark.table("sagar_cat_project1.bronze.iot_telemetry")

# Handle nulls (example: replace missing Product/DeviceType with "Unknown")
df_sales = df_sales.fillna({"Product": "Unknown"})
df_iot = df_iot.fillna({"DeviceStatus": "UNKNOWN"})

# Deduplicate
df_sales = df_sales.dropDuplicates(["OrderID"])
df_iot = df_iot.dropDuplicates(["OrderID", "DeviceType", "OrderTime"])

df_iot: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 12 more fields]
df_sales: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 12 more fields]
```

Normalizing storeid and product categories->



The screenshot shows a Databricks workspace with a notebook titled 'tred_d4'. The notebook contains the following code:

```
df_sales: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 12 more fields]

import pyspark.sql.functions as F
from itertools import chain

df_final = df_joined.withColumn(
    "AnomalyFlag_Sales",
    F.when(F.col("i.AnomalyFlag") == 1, 1).otherwise(0)
)

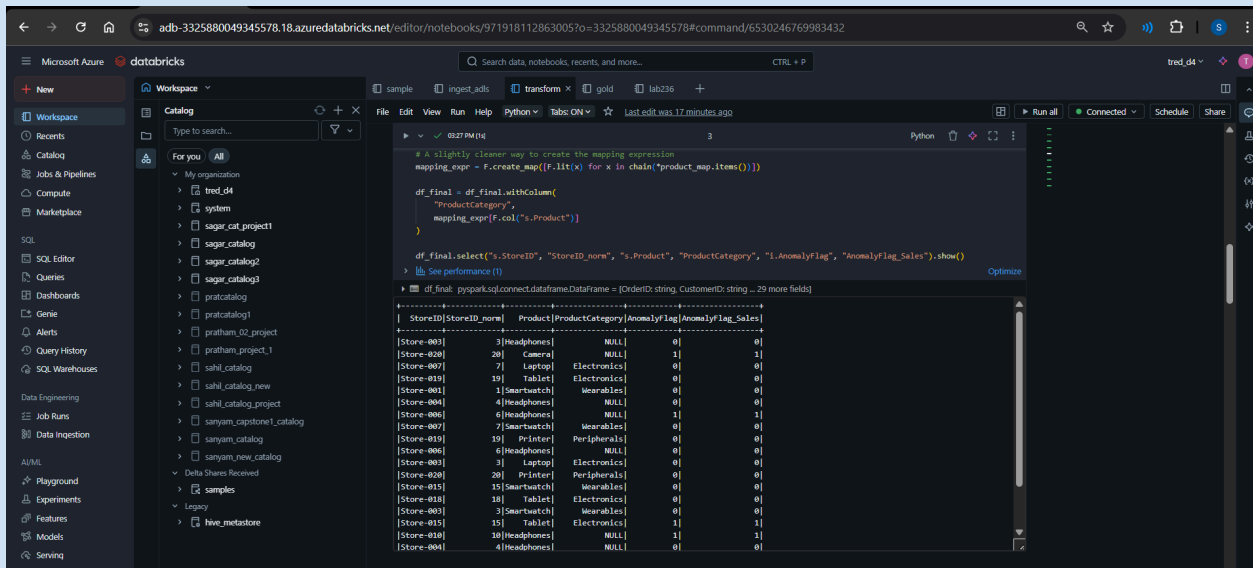
# 2. Normalize store IDs (e.g., "Store-001" -> 1)
# CORRECTED: F.regexp_replace requires a Column object (F.col(...)) as its first argument.
df_final = df_final.withColumn(
    "storeId_norm",
    F.regexp_replace(F.col("s.StoreID"), "Store-", "").cast("int")
)

# 3. Normalize product categories
# IMPROVED: The map should only contain products from the sales data ('s.Product').
# Mixing in device types would result in nulls for actual products.
product_map = {
    "Mobile": "Electronics",
    "Laptop": "Electronics",
    "Tablet": "Electronics",
    "Printer": "Peripherals",
    "Smartwatch": "Wearables"
}

# A slightly cleaner way to create the mapping expression
mapping_expr = F.create_map([F.lit(x) for x in chain("product_map.items()")])

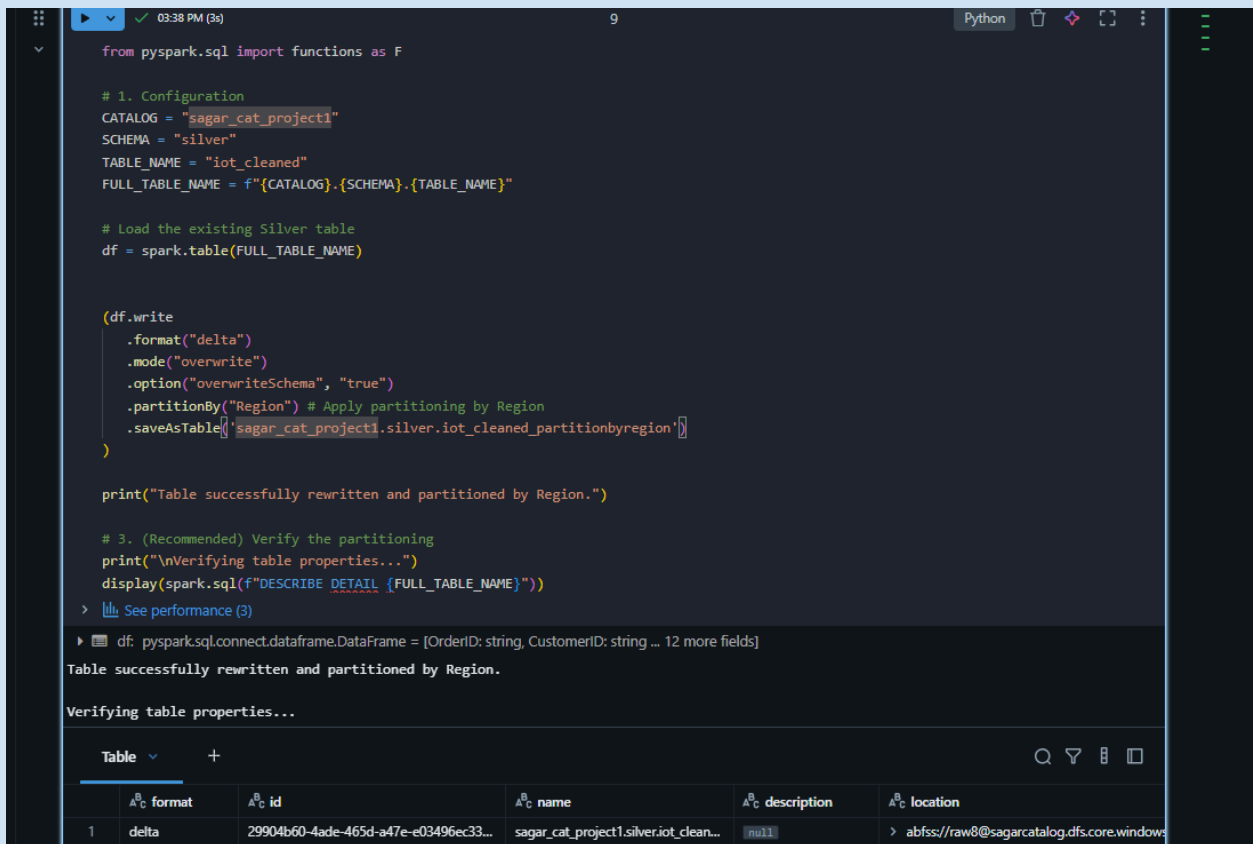
df_final = df_final.withColumn(
    "ProductCategory",
    mapping_expr[F.col("s.Product")]
)

df_final.select("s.StoreID", "storeId_norm", "s.Product", "ProductCategory", "i.AnomalyFlag", "AnomalyFlag_Sales").show()
```



Finally saving in silver schema of my catalog

Partitioning by region and zordering->



Have a look on all the tables that is created in silver region

iot_cleaned->

display(spark.table(full_table_name).limit(10))

--- Displaying top 5 rows of: sagar_cat_project.silver.iot_cleaned ---

OrderID	CustomerID	StoreID	ProductCategory	Region	Quantity	UnitPrice	SalesAmount	OrderTime	DeviceType	Temperature	DeviceStatus	AnomalyFlag	Sales	AnomalyType
6150279-9aef-4381-9489-3c5f09da094	CUST-9476	3	Wearables	East	3	833.22	2499.66	2023-01-08T23:27:07.000+00...	POS_Terminal	18.43	OK	0		
9140269-91ca-4899-801d-cd33085caab	CUST-2373	20	Wearables	North	9	552.72	4974.48	2023-01-01T06:22:03.000+00...	Temp_Sensor	23.21	MISMATCH	1		Rfid_Mismatch
010e09d-61aa-4969-aee5-1317b371c7...	CUST-2860	7	Electronics	West	2	1604.24	3208.48	2023-01-16T01:46:21.000+00...	POS_Terminal	21.4	OK	0		
7623306-584c-4d3d-0993-2071b10e0b...	CUST-5044	19	Electronics	North	4	952.26	3809.04	2023-01-02T21:52:13.000+00...	Temp_Sensor	18.64	OK	0		
28b3795-0556-42e9-8955-c671ca9f88ba	CUST-4842	1	Wearables	East	1	1818.42	1818.42	2023-01-28T20:56:11.000+00...	Temp_Sensor	21.46	OK	0		
10dc64-15db-4c2f-a6d5-5821dc9d3d74	CUST-2843	4	Wearables	West	1	447.28	447.28	2023-01-14T19:01:10.000+00...	Rfid_Scanner	23.84	OK	0		
044c246-5477-4396-8047-5832c5d992	CUST-2650	6	Wearables	South	10	736.08	7360.8	2023-01-15T18:36:04.000+00...	Rfid_Scanner	18.68	MISMATCH	1		Rfid_Mismatch
7714655-5a6d-4d29-8041-d6d97964a4a	CUST-2689	7	Wearables	West	1	945.77	945.77	2023-01-26T03:34:33.000+00...	Rfid_Scanner	18.03	OK	0		
8772d80-246a-46a2-0786-678d6d1010b	CUST-3666	10	Peripherals	North	3	1125.51	3376.53	2023-01-30T11:45:01.000+00...	Rfid_Scanner	24.38	OK	0		
bc9bc732-838f-4153-6896-768357a33c...	CUST-1472	6	Wearables	West	5	76.67	383.35	2023-01-20T19:20:36.000+00...	Rfid_Scanner	19.57	OK	0		

--- Displaying top 5 rows of: sagar_cat_project.silver.iot_cleaned_partitionbyregion ---

after partitioning->

--- Displaying top 5 rows of: sagar_cat_project.silver.iot_cleaned_partitionbyregion ---

OrderID	CustomerID	StoreID	ProductCategory	Region	Quantity	UnitPrice	SalesAmount	OrderTime	DeviceType	Temperature	DeviceStatus	AnomalyFlag	Sales	AnomalyType
010e09d-61aa-4969-aee5-1317b371c7...	CUST-2860		Electronics	West	2	1604.24	3208.48	2023-01-16T01:46:21.000+00...	POS_Terminal	21.4	OK	0		
10dc64-15db-4c2f-a6d5-5821dc9d3d74	CUST-2843		Wearables	West	1	447.28	447.28	2023-01-14T19:01:10.000+00...	Rfid_Scanner	23.84	OK	0		
7714655-5a6d-4d29-8041-d6d97964a4a	CUST-2689		Wearables	West	1	945.77	945.77	2023-01-26T03:34:33.000+00...	Rfid_Scanner	18.03	OK	0		
bc9bc732-838f-4153-6896-768357a33c...	CUST-1472		Wearables	West	5	76.67	383.35	2023-01-20T19:20:36.000+00...	Rfid_Scanner	19.57	OK	0		
a104a6b-14c9-43ce-8d09-ba900d030140	CUST-5436		Electronics	West	1	305.26	305.26	2023-01-29T04:11:45.000+00...	Rfid_Scanner	23.95	OK	0		
e6b5989-8152-418f-9666-ac510777e61	CUST-4259		Wearables	West	10	1362.73	13627.3	2023-01-24T15:58:16.000+00...	Temp_Sensor	21.13	MISMATCH	1		Rfid_Mismatch
2be1ee5-8b22-4c3d-895d-dbe11940d020	CUST-4610		Electronics	West	3	1057.18	3171.54	2023-01-01T11:19:42.000+00...	Temp_Sensor	23.83	OK	0		
307b70c-128c-46e6-4d5d-89a1f0c0ea3	CUST-4470		Electronics	West	3	904.3	2712.9	2023-01-26T03:01:10.000+00...	POS_Terminal	18.85	OK	0		
451212d-6a42-49c5-49c4-576d676275...	CUST-5056		Wearables	West	1	1825.26	1825.26	2023-01-04T04:18:47.000+00...	POS_Terminal	18.8	OK	0		
ad781a3-17be-4845-9648-93a372a6f532	CUST-2946		Electronics	West	3	1600.05	4800.15	2023-01-17T11:03:46.000+00...	Temp_Sensor	24.57	OK	0		

partitioning zorder->

--- Displaying top 5 rows of: sagar_cat_project.silver.iot_cleaned_zorder ---

OrderID	CustomerID	StoreID	ProductCategory	Region	Quantity	UnitPrice	SalesAmount	OrderTime	DeviceType	Temperature	DeviceStatus	AnomalyFlag	Sales	AnomalyType
323c3d1e-15ec-4ff5-bc9d-18edf7000ed	CUST-5643	2	Wearables	West	3	652.34	1957.02	2023-01-06T23:50:45.000+00...	POS_Terminal	24.98	OK	0		
ac3193e5-8676-48a0-bdad-097ab3e08b...	CUST-4066	6	Electronics	West	5	555.67	2778.35	2023-01-06T03:53:03.000+00...	POS_Terminal	35.33	OVERHEAT	1		Overheat
9320599-7804-415c-bc5d-0274d78e70a3	CUST-3960	1	Wearables	West	4	1611	6444	2023-01-06T23:12:08.000+00...	Rfid_Scanner	18.9	OK	0		
4210a853-0e66-4d09-915c-c560837981...	CUST-3598	17	Electronics	West	1	708.58	708.58	2023-01-06T12:47:01.000+00...	Temp_Sensor	19.99	OK	0		
b7cda6b-a05b-4d05-aed5-958d734a2d...	CUST-5748	19	Wearables	West	13	710.43	9235.59	2023-01-06T10:51:25.000+00...	Temp_Sensor	21.35	MISMATCH	1		Rfid_Mismatch
e761c16-5ac1-484f-9707-954472d3d54	CUST-2276	18	Peripherals	West	4	475.85	1903.4	2023-01-06T03:47:23.000+00...	POS_Terminal	23.01	OK	0		
e139a6e-a16d-4779-82b1-c067ceee83...	CUST-5291	15	Wearables	West	4	1143.77	4579.08	2023-01-06T10:20:03.000+00...	Rfid_Scanner	24.62	OK	0		
716a485-79e9-418a-418a-5a5970c2a8d	CUST-5473	6	Peripherals	West	2	364.85	729.7	2023-01-06T16:13:06.000+00...	Rfid_Scanner	24.96	OK	0		
2a3a6d3-1388-49a2-0863-2818d50549b	CUST-1001	20	Wearables	West	1	1430.75	1430.75	2023-01-06T19:47:08.000+00...	Rfid_Scanner	24	OK	0		
e6569d7-c309-4376-836e-c1d5d53d09e	CUST-8405	3	Electronics	North	4	220.25	881	2023-01-19T23:10:41.000+00...	Rfid_Scanner	20.02	OK	0		

Aggregation & Enrichment Layer (Gold)->

from pyspark.sql import functions as F

from pyspark.sql.window import Window

CATALOG = "sagar_cat_project1"

SILVER_SCHEMA = f"{CATALOG}.silver"

GOLD_SCHEMA = f"{CATALOG}.gold"

BASE_TABLE = f"{SILVER_SCHEMA}.iot_cleaned"

Create the Gold schema if it doesn't exist

spark.sql(f"CREATE SCHEMA IF NOT EXISTS {GOLD_SCHEMA}")

```

# Load the base Silver table
base_df = spark.table(BASE_TABLE)

# Standardize column names to ensure consistency
# (Handles cases like 'sales_amount' vs 'SalesAmount' and 'StoreID_norm' vs 'StoreID')
if "sales_amount" in base_df.columns and "SalesAmount" not in base_df.columns:
    base_df = base_df.withColumnRenamed("sales_amount", "SalesAmount")
if "StoreID_norm" in base_df.columns and "StoreID" not in base_df.columns:
    base_df = base_df.withColumnRenamed("StoreID_norm", "StoreID")

# Ensure correct data types for calculations
base_df = (base_df
    .withColumn("OrderTime", F.to_timestamp("OrderTime"))
    .withColumn("SalesAmount", F.col("SalesAmount").cast("double"))
    .withColumn("Quantity", F.col("Quantity").cast("int"))
)

# Derive common date and month grains for aggregation
base_df = (base_df
    .withColumn("order_date", F.to_date("OrderTime"))
    .withColumn("order_month", F.date_trunc("month", "OrderTime"))
)

# -----
# 1) KPI: Daily and Monthly Sales per Region
# -----
print("Creating sales KPIs per region...")

# Daily aggregation
daily_sales = (base_df
    .groupBy("order_date", "Region")
    .agg(
        F.sum("SalesAmount").alias("total_sales"),
        F.sum("Quantity").alias("total_units_sold"),
        F.countDistinct("OrderID").alias("distinct_orders")
    )
)

```

```

    )
)
(daily_sales.write.format("delta").mode("overwrite")
  .saveAsTable(f"{GOLD_SCHEMA}.daily_sales_by_region"))

# Monthly aggregation
monthly_sales = (base_df
  .groupBy("order_month", "Region")
  .agg(
    F.sum("SalesAmount").alias("total_sales"),
    F.sum("Quantity").alias("total_units_sold"),
    F.countDistinct("OrderID").alias("distinct_orders")
  )
)
(monthly_sales.write.format("delta").mode("overwrite")
  .saveAsTable(f"{GOLD_SCHEMA}.monthly_sales_by_region"))

# -----
# 2) KPI: Device Anomaly Trend per Store
# -----
print("Creating device anomaly trend KPI...")

anomaly_trend = (base_df
  .groupBy("order_date", "StoreID")
  .agg(
    F.sum("AnomalyFlag_Sales").alias("anomaly_count"),
    F.count("OrderID").alias("total_transactions")
  )
  .withColumn("anomaly_rate", F.col("anomaly_count") / F.col("total_transactions"))
)
(anomaly_trend.write.format("delta").mode("overwrite")
  .saveAsTable(f"{GOLD_SCHEMA}.daily_anomaly_trend_by_store"))

# -----
# 3) KPI: Conversion Rate Impact from Device Failures
# -----
print("Creating conversion rate impact KPI...")

```

```

# We define "conversion" as the ratio of successful orders to total orders at a store-day level
daily_impact = (base_df
    .groupBy("order_date", "StoreID")
    .agg(
        F.sum(F.when(F.col("AnomalyFlag_Sales") == 1, 1).otherwise(0)).alias("anomaly_orders"),
        F.count("OrderID").alias("total_orders")
    )
    .withColumn("successful_orders", F.col("total_orders") - F.col("anomaly_orders"))
    .withColumn("success_rate", F.col("successful_orders") / F.col("total_orders"))
    .withColumn("had_anomaly_day", F.when(F.col("anomaly_orders") > 0, 1).otherwise(0))
)
(daily_impact.write.format("delta").mode("overwrite")
    .saveAsTable(f"{GOLD_SCHEMA}.daily_conversion_impact_by_store"))

# -----
# 4) Enrichment: Tag Stores into Performance Tiers
# -----
print("Enriching stores with performance tiers...")

# Calculate total sales per store for each month
store_monthly_sales = base_df.groupBy("order_month",
    "StoreID").agg(F.sum("SalesAmount").alias("total_sales"))

# Use a window function to rank stores within each month
w_spec = Window.partitionBy("order_month").orderBy(F.col("total_sales").desc())
store_ranks = store_monthly_sales.withColumn("rank", F.percent_rank().over(w_spec))

# Assign tiers based on percentile rank
store_tiers = store_ranks.withColumn("performance_tier",
    F.when(F.col("rank") <= 0.2, "High")
    .when((F.col("rank") > 0.2) & (F.col("rank") <= 0.8), "Medium")
    .otherwise("Low")
)
(store_tiers.select("order_month", "StoreID", "total_sales", "performance_tier")
    .write.format("delta").mode("overwrite")
    .saveAsTable(f"{GOLD_SCHEMA}.monthly_store_performance_tiers"))

# -----

```

```

# 5) Enrichment: Compute Weighted Sales vs. Anomalies Score
# -----

print("Enriching stores with a weighted performance score...")

# Get monthly sales and anomaly counts per store
monthly_kpis = base_df.groupBy("order_month", "StoreID").agg(
    F.sum("SalesAmount").alias("sales"),
    F.sum("AnomalyFlag_Sales").alias("anomalies")
)

# Define a window to normalize scores across all stores within a month
w_monthly = Window.partitionBy("order_month")

# Calculate min-max normalized scores for sales and anomalies
normalized_kpis = (monthly_kpis
    .withColumn("max_sales", F.max("sales").over(w_monthly))
    .withColumn("min_sales", F.min("sales").over(w_monthly))
    .withColumn("max_anomalies", F.max("anomalies").over(w_monthly))
    .withColumn("min_anomalies", F.min("anomalies").over(w_monthly))
    .withColumn("sales_norm",
        F.when(F.col("max_sales") == F.col("min_sales"), 1.0)
        .otherwise((F.col("sales") - F.col("min_sales")) / (F.col("max_sales") - F.col("min_sales"))))
    )
    .withColumn("anomalies_norm",
        F.when(F.col("max_anomalies") == F.col("min_anomalies"), 0.0)
        .otherwise((F.col("anomalies") - F.col("min_anomalies")) / (F.col("max_anomalies") -
F.col("min_anomalies"))))
    )
)

# Compute the final weighted score (e.g., 80% weight for sales, 20% for anomalies)
weighted_score = normalized_kpis.withColumn(
    "performance_score",
    (0.8 * F.col("sales_norm")) - (0.2 * F.col("anomalies_norm"))
)

(weighted_score.select("order_month", "StoreID", "sales", "anomalies", "performance_score")
    .write.format("delta").mode("overwrite")
)

```

```
.saveAsTable(f"{GOLD_SCHEMA}.monthly_store_weighted_score"))

print("\n--- Gold Layer Generation Complete ---")
```

Finally have a look of all the tables that is created.

The screenshot shows the Databricks workspace interface. On the left is a sidebar with navigation options like Workspace, Catalog, and Recent. The main area displays a Python notebook with the following code:

```
full_table_name = f"{GOLD_SCHEMA}.{table_name}"
print(f"--- Displaying contents of: {full_table_name} ---")

# Use display() for a rich, interactive table view in Databricks notebooks
display(spark.table(full_table_name).head(5))
```

Below the code, the output shows the contents of the table 'sagar_cat_project1.gold.daily_sales_by_region'.

order_id	order_date	Region	1.2 total_sales	1.3 total_units_sold	1.4 distinct_orders
1	2023-01-30	East	30215.19999999999...	20	7
2	2023-01-29	South	32773.439999999999	28	11
3	2023-01-02	East	27086.73	19	7
4	2023-01-11	North	29061.52000000000...	36	11
5	2023-01-11	East	24138.62999999999...	25	9

The table view indicates 5 rows and a runtime of 5.16s. It was refreshed 45 minutes ago.

Below this, the contents of 'sagar_cat_project1.gold.monthly_sales_by_region' are displayed:

order_month	Region	1.2 total_sales	1.3 total_units_sold	1.4 distinct_orders
2023-01-01T00:00:00.000-00...	East	776331.6000000001	811	252
2023-01-01T00:00:00.000-00...	South	704848.18	738	225
2023-01-01T00:00:00.000-00...	North	955183.64	960	289
2023-01-01T00:00:00.000-00...	West	769714.7799999996	787	234

This table view indicates 4 rows and a runtime of 5.16s. It was also refreshed 45 minutes ago.

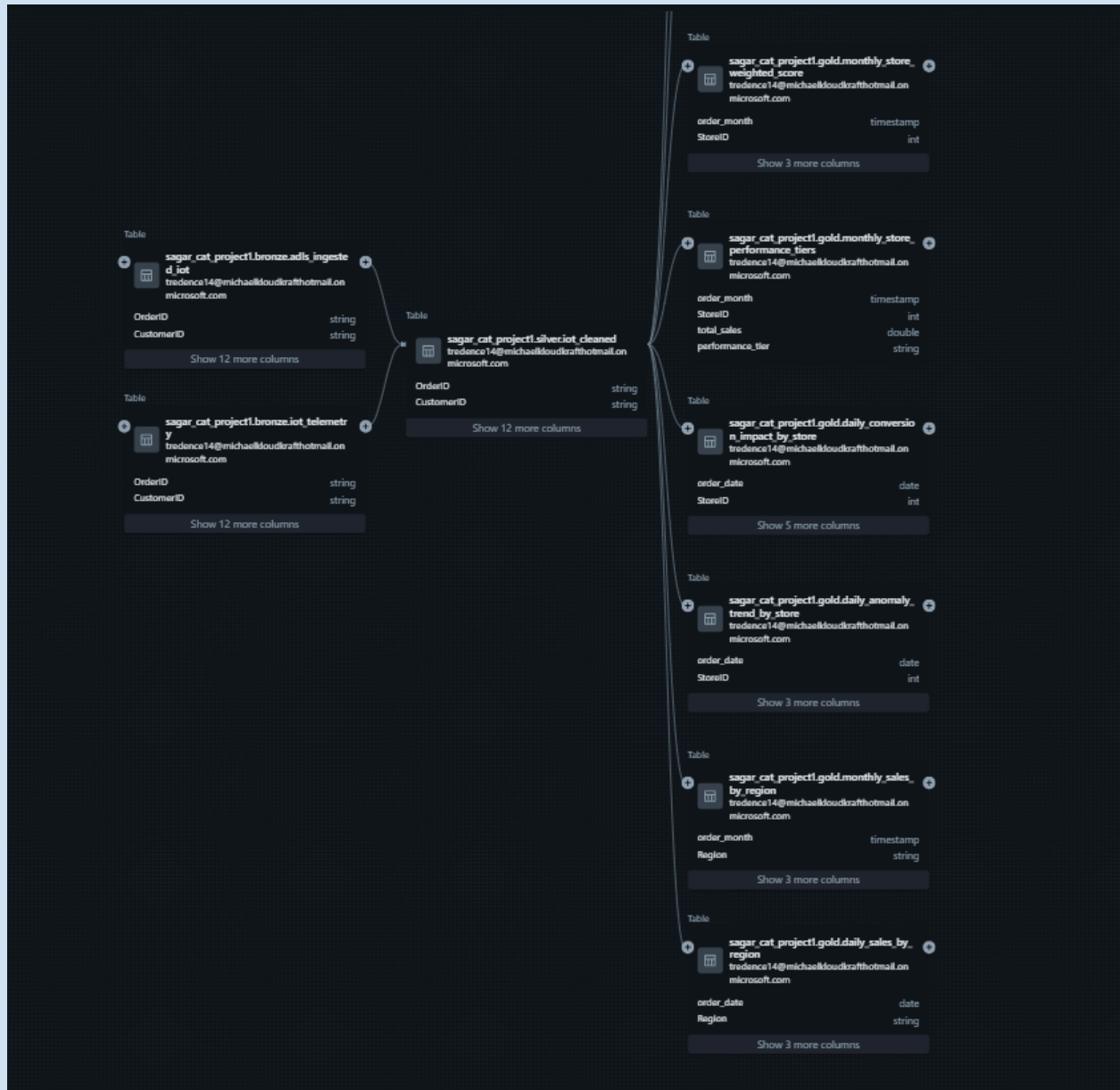
JOBS creation->

The screenshot displays the Databricks Jobs & Pipelines interface. The main view shows a job named 'sagar_cap1 run' in the 'List' view. The job consists of three steps: 'Ingest', 'transform', and 'Gold', all of which have succeeded. The 'Job run details' panel on the right shows the job ID, run ID, launch time, and status (Succeeded). The 'Compute' panel shows the job is running on a 'kraft200' node.

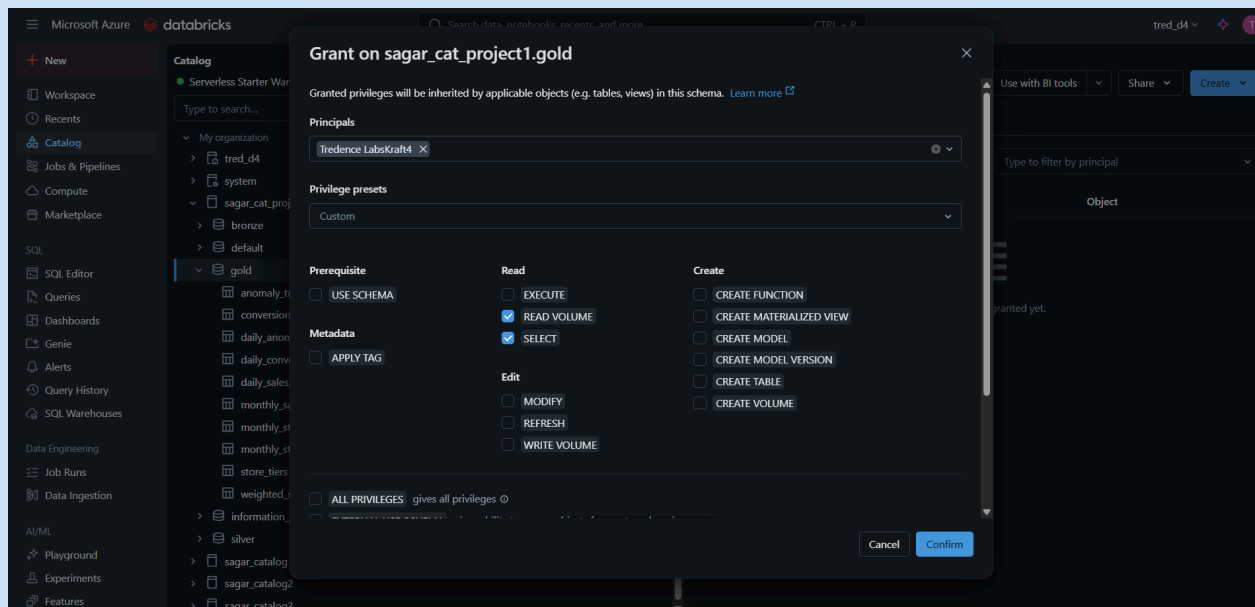
scheduling for regular ingestion and updation of tables->

The screenshot displays the Databricks Jobs & Pipelines interface with the 'Schedules & Triggers' dialog open for a job named 'sagar_cap1'. The dialog is configured with 'Trigger Status' set to 'Active', 'Trigger type' set to 'Scheduled', and 'Schedule type' set to 'Simple'. The 'Periodic' schedule is configured with 'Every 1 Day'. The 'Run now' button is visible in the top right corner of the job details panel.

Lineage Graph->



Giving bi user access to my gold tables->



Connecting through PowerBi->

